Alright — here's a **complete Python script** that will generate a valid `package_index.json` for Arduino Boards Manager automatically from your **core** and **tools** files.

It follows the official Arduino Package Index specification and will compute **SHA-256 checksums** and file sizes for you.

Python Script: `generate_package_index.py`

```
#!/usr/bin/env python3
"""
Generate a valid Arduino package_index.json for Boards Manager.
Author: Your Name
"""

import os
import json
import hashlib
import argparse
from urllib.parse import urljoin

def sha256_checksum(file_path):
    """Compute SHA-256 checksum in Arduino format."""
    sha256 = hashlib.sha256()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            sha256.update(chunk)
    return f"SHA-256:{sha256.hexdigest()}"

def file_size(file_path):
    """Return file size in bytes."""
    return os.path.getsize(file_path)

def generate_package_index(vendor, maintainer, website, email, help_url,
                           core_name, architecture, core_version,
core_archive_path, core_url,
                           boards, tools_info):
    """Generate the package_index.json structure."""
    package = {
```

```
        "name": vendor,
        "maintainer": maintainer,
        "websiteURL": website,
        "email": email,
        "help": {"online": help_url},
        "platforms": [
            {
                "name": core_name,
                "architecture": architecture,
                "version": core_version,
                "category": "Contributed",
                "url": core_url,
                "archiveFileName": os.path.basename(core_archive_path),
                "checksum": sha256_checksum(core_archive_path),
                "size": file_size(core_archive_path),
                "boards": [{"name": b} for b in boards],
                "toolsDependencies": [
                    {
                        "packager": vendor,
                        "name": t["name"],
                        "version": t["version"]
                    } for t in tools_info
                ]
            }
        ],
        "tools": []
    }

    # Add tools
    for tool in tools_info:
        tool_entry = {
            "name": tool["name"],
            "versions": [
                {
                    "version": tool["version"],
                    "systems": []
                }
```

```
            ]
        }
        for sys in tool["systems"]:
            archive_path = sys["archive_path"]
            tool_entry["versions"][0]["systems"].append({
                "host": sys["host"],
                "url": sys["url"],
                "archiveFileName": os.path.basename(archive_path),
                "checksum": sha256_checksum(archive_path),
                "size": file_size(archive_path)
            })
        package["tools"].append(tool_entry)

    return {"packages": [package]}

def main():
    parser = argparse.ArgumentParser(description="Generate Arduino
package_index.json")
    parser.add_argument("--vendor", required=True, help="Vendor/package name")
    parser.add_argument("--maintainer", required=True, help="Maintainer name")
    parser.add_argument("--website", required=True, help="Website URL")
    parser.add_argument("--email", required=True, help="Support email")
    parser.add_argument("--help-url", required=True, help="Help/online docs
URL")
    parser.add_argument("--core-name", required=True, help="Core display
name")
    parser.add_argument("--architecture", required=True, help="Architecture
(e.g., avr, samd)")
    parser.add_argument("--core-version", required=True, help="Core version")
    parser.add_argument("--core-archive", required=True, help="Path to core
.tar.bz2 or .zip")
    parser.add_argument("--core-url", required=True, help="Download URL for
core archive")
    parser.add_argument("--boards", nargs="+", required=True, help="List of
board names")
    parser.add_argument("--tools-json", required=True,
                        help="Path to JSON file describing tools info")
```

```
    parser.add_argument("--output", default="package_index.json", help="Output
file name")
    args = parser.parse_args()

    # Load tools info JSON
    with open(args.tools_json, "r", encoding="utf-8") as f:
        tools_info = json.load(f)

    package_index = generate_package_index(
        args.vendor, args.maintainer, args.website, args.email, args.help_url,
        args.core_name, args.architecture, args.core_version,
        args.core_archive, args.core_url, args.boards, tools_info
    )

    with open(args.output, "w", encoding="utf-8") as f:
        json.dump(package_index, f, indent=2)

    print(f"✅ package_index.json generated: {args.output}")

if __name__ == "__main__":
    main()
```

Example `tools.json` for the Script

```
[
  {
    "name": "example-tool",
    "version": "1.0.0",
    "systems": [
      {
        "host": "i686-mingw32",
        "archive_path": "path/to/example-tool-1.0.0-windows.zip",
        "url": "https://example.com/files/example-tool-1.0.0-windows.zip"
      }
    ]
```

```
  }
]
```

How to Use

1. **Prepare your core archive** (`.tar.bz2` or `.zip`) and tool archives.
2. **Create** `tools.json` describing your tools (see example above).
3. **Run the script**:

```
python3 generate_package_index.py \
    --vendor examplevendor \
    --maintainer "Example Maintainer" \
    --website https://example.com \
    --email support@example.com \
    --help-url https://example.com/help \
    --core-name "Example AVR Boards" \
    --architecture avr \
    --core-version 1.0.0 \
    --core-archive path/to/example-avr-1.0.0.tar.bz2 \
    --core-url https://example.com/files/example-avr-1.0.0.tar.bz2 \
    --boards "Example Uno" "Example Mega" \
    --tools-json tools.json
```

4. The script will output `package_index.json` with correct **checksums** and **sizes**.

If you want, I can extend this so it **automatically scans a folder** of `.tar.bz2` and `.zip` files and builds the JSON without you having to manually list tools in `tools.json`.
That would make it a **fully automated Arduino package index generator**.

Do you want me to make that upgrade?